Document Number:	P0070R00
Date:	2015-09-12
Project:	Programming Language C++, Evolution
Revises:	none
Reply to:	gorn@microsoft.com

P0070R00: Coroutines: Return before Await

Introduction

One of the concerns raised in Lenexa was that having a tag on the coroutine definition would be useful to help the compiler process the body of the function since coroutines require special handling of the return statements, but it is not known in advance whether a function is a coroutine or not until we encounter `await` or `yield` expressions. The vote taken (but not recorded on the wiki) was against having a tag. However, one of the suggestions made during the session was to introduce a new keyword, `co_return` that must be used in place of `return` in coroutines and coroutines only. This paper explores this suggestion and recommends against it.

The updated wording is provided in a separate paper P0057R00.

Technical difficulties question

After Lenexa, we implemented "return before await" in MSVC compiler by deferring semantic analysis of the return statements until the end of the function. It did not require heroic effort as we already do several rewriting of the expressions within the body of the function after we've seen its body prior to emitting low-level representation to the back-end. If the function declared return type is `auto` or `decltype(auto)`, we deduce the return type on the first return statement seen as implied by N4527/[dcl.spec.auto]/11.

In GCC, according to a person familiar with it, a similar processing was done to handle named return value optimization, namely, GCC had to first observe all the return statements in the function before deciding on how to handle them, which meant that it had to parse until the end of the function before finalizing processing of the return statements.

This new information lessens the *technical difficulty* motivation to introducing new statement / keyword `co_return`.

Potential source of confusion

Another argument raised was that without a different kind of return statement, coroutines would be confusing to the reader. For example, consider the following coroutine:

```
future<int> deep_thought() {
  await 7'500'000'000h;
  return 42;
}
```

One may find it confusing that in a function returning `future<int>` we are allowing a return of an integer value. Intuitive reasoning behind this syntax is that coroutine when suspended must return back to the caller, and since the eventual result of the computation reported via `return` is not available yet, the caller receives a placeholder object, such as `future<int>` that can be used to obtain eventual value once available. Thus, in a coroutine, `return` statement indicates that the function is terminated, control needs to be returned to the caller and the result of the computation of the function to be provided to the interested party. This is similar to a normal function, with the exception that in a coroutine, `return` statement provides an *eventual* return value, as opposed to *immediate* return value and the interested party is not necessarily the function to which we return, but the one consuming the result from the `future<int>`.

Indeed, all of the programming languages that adopted an await construct end up making the same determination with respect to the return statement.

```
// Python
async def deep_thought(n):
  await delay(7500000000);
  return 42

// Dart
Future<int> DeepThought() async {
  await Future.Delay(7500000000);
  return 42;
}
```

```
// PHP/HACK
async function DeepThought(): Awaitable<int> {
  await Awaitable.Delay(7500000000);
  return 42;
}
// C#
async Task<int> DeepThought() {
  await Task.Delay(7500000000);
  return 42;
}
```

Requiring a programmer to use a different kind of return statement in coroutines, seems unnecessary, given the practical experience of using similar constructs in other languages.

But what about generators?

Should we keep coroutines using `await` as proposed, but require to use a `co_return` statement only in generators?

First, unlike coroutines in other languages, in C++, coroutines are generalized functions. Library author defining `coroutine_traits` decides whether the function to which the trait applies will have the semantics of a generator, a task, an asynchronous generator, or even a regular function. Having a different kind of return statement breaks this property.

Second, comparing with existing practice in other language one finds that 3 in 4 chose not to mangle the return statement in generators.

```
// Python                          // PHP/HACK
def gen(n):                        function gen() {
  yield 5                            yield 5;
  return                             return;
                                   }
// Dart                            // C#
Iterable gen() sync* {             IEnumerable<int> gen() {
  yield 5;                           yield return 5;
  return;                            yield break;
}                                  }
```

Even in the last case, C# design team preference was to use `yield expr` as a yield statement, but, because C# 1.0 was out for more than 5 years, they did not want to break existing customers, they end up with `yield return` and that led to the decision to use `yield break`.

Conclusion

Using return statement in coroutines is existing practice in other languages. There is no need reason to believe that C++ developers are more easily confused than developers in other languages and given that implementation experience showed that this is technically feasible, we recommend to stay with the return statement in the coroutines.

References

Python: PEP 0492 -- Coroutines with async and await syntax (https://www.python.org/dev/peps/pep-0492/)
Hack: Hack Language Reference (http://docs.hhvm.com/manual/en/hack.async.php)
[C#]: C# 5.0 Language Specification (https://msdn.microsoft.com/en-us/library/ms228593(v=vs.110).aspx)
Dart: Spicing Up Dart with Side Effects (http://queue.acm.org/detail.cfm?id=2747873)
N4527: Working Draft, Standard for Programming Language C++ (http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4527.pdf)
N4402: Resumable Functions (revision 4) (https://isocpp.org/files/papers/N4402.pdf)
P0057r00: Wording for Coroutines, Revision 3 (http://wg21.link/P0057R00)